

PH3120

Computational Physics

Laboratory 1

CPL104 – Vectors and
Matrices Section 1 & 2

1. I. (a)

Vector	Scalar quantity
Physical quantity that is completely described by both its magnitude and direction.	Physical quantity that is completely described by magnitude alone.

(b) if,

$$\vec{V} = x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$$

Magnitude of $\vec{V}$ is,

$$|\vec{V}| = \sqrt{X^2 + Y^2 + Z^2}$$

(c) $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}|\cos\theta$

(d) $\vec{a} \times \vec{b} = |\vec{a}||\vec{b}|\sin\theta \times \hat{n}$

(e) $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}|\cos\theta$
 $\cos\theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|}$

So,

$$\theta = \cos^{-1} \left(\frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|} \right)$$

(f) Consider, $\overrightarrow{AB} = \mathbf{a}$

Then, $|\overrightarrow{AB}| = |\mathbf{a}|$

If unit vector of $\overrightarrow{AB} = \hat{u}$

$$\hat{u} = \frac{\overrightarrow{AB}}{|\overrightarrow{AB}|}$$

II. (a)

i

```
import numpy as np

#k=1/4*pi*E0
k = 8.9875e9
Q1 = 2e-6
Q2 = -4e-6
Q3 = 6e-6

r1_charge = np.array([1.0, 0.0])
r2_charge = np.array([0.0, 2.0])
r3_charge = np.array([-2.0, -1.0])

r_origin = np.array([0.0, 0.0])

print("\nElectric Field due to Q1")
d1 = r_origin - r1_charge
mag_d1 = np.linalg.norm(d1)
unit_d1 = d1 / mag_d1
E1 = k * Q1 / (mag_d1**2) * unit_d1
print(f"Displacement vector for Q1 to origin (d1): {d1} m")
print(f"Distance from Q1 to origin (mag_d1): {mag_d1} m")
print(f"Unit vector (hat_d1): {unit_d1}")
print(f"Electric field E1: {E1} N/C")

print("\nElectric Field due to Q2")
d2 = r_origin - r2_charge
mag_d2 = np.linalg.norm(d2)
unit_d2 = d2 / mag_d2
E2 = k * Q2 / (mag_d2**2) * unit_d2
print(f"Displacement vector for Q2 to origin (d2): {d2} m")
print(f"Distance from Q2 to origin (mag_d2): {mag_d2} m")
```

```

mag_d2 = np.linalg.norm(d2)
unit_d2 = d2 / mag_d2
E2 = k * Q2 / (mag_d2**2) * unit_d2
print(f"Displacement vector for Q2 to origin (d2): {d2} m")
print(f"Distance from Q2 to origin (mag_d2): {mag_d2} m")
print(f"Unit vector (hat_d2): {unit_d2}")
print(f"Electric field E2: {E2} N/C")

print("\nElectric Field due to Q3")
d3 = r_origin - r3_charge
mag_d3 = np.linalg.norm(d3)
unit_d3 = d3 / mag_d3
E3 = k * Q3 / (mag_d3**2) * unit_d3
print(f"Displacement vector for Q3 to origin (d3): {d3} m")
print(f"Distance from Q3 to origin (mag_d3): {mag_d3} m (sqrt(5) approx {np.sqrt(5):.4f})")
print(f"Unit vector (hat_d3): {unit_d3}")
print(f"Electric field E3: {E3} N/C")

```

```

Electric Field due to Q1
Displacement vector for Q1 to origin (d1): [-1.  0.] m
Distance from Q1 to origin (mag_d1): 1.0 m
Unit vector (hat_d1): [-1.  0.]
Electric field E1: [-17975.  0.] N/C

```

```

Electric Field due to Q2
Displacement vector for Q2 to origin (d2): [ 0. -2.] m
Distance from Q2 to origin (mag_d2): 2.0 m
Unit vector (hat_d2): [ 0. -1.]
Electric field E2: [ -0.  8987.5] N/C

```

```

Electric Field due to Q3
Displacement vector for Q3 to origin (d3): [2. 1.] m
Distance from Q3 to origin (mag_d3): 2.23606797749979 m (sqrt(5) approx 2.2361)
Unit vector (hat_d3): [0.89442719 0.4472136 ]
Electric field E3: [9646.39725493 4823.19862747] N/C

```

```
import numpy as np
```

```
#k=1/4*pi*E0
```

```
k = 8.9875e9
```

```
Q1 = 2e-6
```

```
Q2 = -4e-6
```

```
Q3 = 6e-6
```

```
r1_charge = np.array([1.0, 0.0])
```

```
r2_charge = np.array([0.0, 2.0])
```

```
r3_charge = np.array([-2.0, -1.0])
```

```
r_origin = np.array([0.0, 0.0])
```

```
d1 = r_origin - r1_charge
```

```
mag_d1 = np.linalg.norm(d1)
```

```
unit_d1 = d1 / mag_d1
```

```
E1 = k * Q1 / (mag_d1**2) * unit_d1
```

```
d2 = r_origin - r2_charge
```

```
mag_d2 = np.linalg.norm(d2)
```

```
unit_d2 = d2 / mag_d2
```

```
E2 = k * Q2 / (mag_d2**2) * unit_d2
```

```
d3 = r_origin - r3_charge
```

```
mag_d3 = np.linalg.norm(d3)
```

```
unit_d3 = d3 / mag_d3
```

```
E3 = k * Q3 / (mag_d3**2) * unit_d3
```

```
E_total = E1 + E2 + E3
```

```
print(f"Total electric field E_total (vector sum E1 + E2 + E3): {E_total} N/C")
```

```
Total electric field E_total (vector sum E1 + E2 + E3): [-8328.60274507 13810.69862747] N/C
```

```
import numpy as np

#k=1/4*pi*E0
k = 8.9875e9
Q1 = 2e-6
Q2 = -4e-6
Q3 = 6e-6

r1_charge = np.array([1.0, 0.0])
r2_charge = np.array([0.0, 2.0])
r3_charge = np.array([-2.0, -1.0])

r_origin = np.array([0.0, 0.0])

d1 = r_origin - r1_charge
mag_d1 = np.linalg.norm(d1)
unit_d1 = d1 / mag_d1
E1 = k * Q1 / (mag_d1**2) * unit_d1

d2 = r_origin - r2_charge
mag_d2 = np.linalg.norm(d2)
unit_d2 = d2 / mag_d2
E2 = k * Q2 / (mag_d2**2) * unit_d2

d3 = r_origin - r3_charge
mag_d3 = np.linalg.norm(d3)
unit_d3 = d3 / mag_d3
E3 = k * Q3 / (mag_d3**2) * unit_d3

E_total = E1 + E2 + E3

mag_E_total = np.linalg.norm(E_total)
print(f"Magnitude of total electric field: {mag_E_total} N/C")
angle_rad = np.arctan2(E_total[1], E_total[0])
angle_deg = np.degrees(angle_rad)
print(f"Direction of total electric field (angle with positive x-axis): {angle_deg:.2f} degrees")

unit_E_total = E_total / mag_E_total
print(f"Unit vector of total electric field: {unit_E_total}")

Magnitude of total electric field: 16127.647697784061 N/C
Direction of total electric field (angle with positive x-axis): 121.09 degrees
Unit vector of total electric field: [-0.5164177  0.85633683]
```

(b)

```
import numpy as np

Angular_velocity = np.array([0, 0, 5])
I_moment = np.array([
    [3, 0, 0],
    [0, 5, 0],
    [0, 0, 4]
])

L = I_moment * Angular_velocity
print(f"Calculated Angular Momentum (L):\n{L} kg m^2/s\n")
```

```
Calculated Angular Momentum (L):
[[ 0  0  0]
 [ 0  0  0]
 [ 0  0 20]] kg m^2/s
```

(c)

```
import numpy as np

Q = 2e-6
Angular_velocity = np.array([3, 4, 0])
Magnetic_field = np.array([0.0, 0.2, 0.3])

Force = Q*np.cross(Angular_velocity, Magnetic_field)
print(f"Calculated magnetic force (Force):\n{Force} N")
```

```
Calculated magnetic force (Force):
[ 2.4e-06 -1.8e-06  1.2e-06] N
```

2. I.

Property	Addition	Multiplication	1. (a) Let A, B and C be real
Commutative	$A + B = B + A$	$A \cdot B = B \cdot A$	
Associative	$(A + B) + C = A + (B + C)$	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	
Identity	$A + 0 = A$ $0 + A = A$	$A \cdot 1 = A$ $1 \cdot A = A$	
Inverse	$A + (-A) = 0$	$A \cdot 1/A = 1 ; A \neq 0$	

numbers,

Also, the Distributive property involves both addition and multiplication.

$$A(B + C) = AB + AC$$

(b) In matrix transposition, the rows of the original matrix become columns of the transposed matrix, and vice versa. Matrix A transposed is denoted as A^T or A' .

$$\text{Ex: } A = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 2 & 3 \\ 0 & 5 & 0 \end{pmatrix}$$

$$A^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 5 \\ 2 & 3 & 0 \end{pmatrix}$$

If the original matrix A has dimensions $m \times n$ (m-rows, n-columns), then its transpose A^T will have dimensions $n \times m$.

(c) If the multiplication of two matrix is equal to the Identity matrix(I), A one matrix is called as another invers.

$$\text{Ex: } A \cdot A^{-1} = I$$

Then, A^{-1} is the inverse matrix of A.

To be the inverse matrix there are some conditions,

- It must be a square matrix.
- Its determinant must not be zero.

(d) Square matrix has the same number of rows and columns. So, it is like perfect square shape.

But in the rectangular matrix, that doesn't have the same number of rows and columns. So, it looks more like rectangle.

Therefore main difference between the square matrix and rectangle matrix is dimension.

(e) In diagonal matrix, the main elements on the main diagonal can be any value. (Zero or non-Zero). The entries outside of the main diagonal are zero.

Let A is 3x3 diagonal matrix,

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 3 \\ 0 & 0 & 0 \end{pmatrix}$$

(f) Identity matrix is denoted by I; it is a special type of square matrix. Its main diagonal has 1s and every other where has 0s.

Properties of identity matrix;

- Square matrix: Identity matrix always should be a square matrix
- Diagonal matrix: Here non diagonal elements are zero. So, Identity matrix is a diagonal matrix
- Multiplicative Identity: $A \cdot I = A$ (A is an $m \times n$ matrix)
- Inverse: The inverse of identity matrix is its own matrix.
- Eigenvalues: All eigenvalues of an identity matrix are 1
- Scaler matrix: Diagonal elements are 1. So, this is a scaler matrix.

When consider about uses Identity matrix in computations,
Mainly use Identity matrix to find inverse.

$$A \cdot A^{-1} = I$$

So, in this equation we can easily find Inverse matrix.

(g) The rank of a matrix is the maximum number of linearly independent rows (or columns) in the matrix. It essentially quantifies the "size" of the matrix in terms of its linearly independent components. The rank is directly related to linear independence: the rank of a matrix is equal to the number of linearly independent rows or columns it contains.

(h) Yes. The rank of the matrix can change the addition; multiplication like under matrix operations.

II.

(a)

```
import numpy as np
def matrix_sum(A, B):

    if A.shape != B.shape:
        print("Error: Matrices must have the same dimensions for addition.")
    return A+B

A = np.array([[5,2], [2,4]])
B = np.array([[9,6], [3,8]])

Sum = matrix_sum(A, B)
print("A+B:\n", Sum)
```

```
A+B:
[[14  8]
 [ 5 12]]
```

(b)

```
import numpy as np
def det_of_matrix(matrix):

    if matrix.shape[0] != matrix.shape[1]:
        print("Must be a square matrix for calculate determinant")
    return np.linalg.det(matrix)

A = np.array([[5, 2], [1, 6]])

det = det_of_matrix(A)
print("Matrix A:\n", A)
print("Determinant of A: ",det)
```

Matrix A:
[[5 2]
[1 6]]
Determinant of A: 27.999999999999996

(c)

```
import numpy as np

def transpose_matrix(A):
    return A.T

A = np.array([[3, 4, 1],
              [1, 7, 4]])
transpose = transpose_matrix(A)
print("Transpose of A: \n",transpose)
```

Transpose of A:
[[3 1]
[4 7]
[1 4]]

(d)

```
import numpy as np

def check_symmetric(A):

    if A.shape[0] != A.shape[1]:
        return False
    return np.array_equal(A, A.T)

A = np.array([[4, 2, 1],
              [2, 8, 5],
              [1, 5, 2]])

print ("Matrix A: \n", A)
print ("Is a symmetric?", check_symmetric(A))
```

```
Matrix A:
[[4 2 1]
 [2 8 5]
 [1 5 2]]
Is a symmetric? True
```

(e)

```
import numpy as np

def matrix_multiply(A, B):

    if A.shape[1] != B.shape[0]:
        print ("Number of columns must be equal")
    return np.dot(A, B)

A = np.array([[3,2],
              [6,1]])
B = np.array([[2,9],
              [6,2]])

A_dot_B = matrix_multiply(A, B)
print ("A.B: \n", A_dot_B)
```

```
A.B:
[[18 31]
 [18 56]]
```

III. (a) Analytical solution:

$$A = X X^T + Y Y^T$$

$$X = \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \quad X^T = \begin{pmatrix} \cos(\theta) & \sin(\theta) \end{pmatrix}$$

$$X X^T = \begin{pmatrix} \cos^2(\theta) & \cos(\theta) \sin(\theta) \\ \cos(\theta) \sin(\theta) & \sin^2(\theta) \end{pmatrix}$$

$$Y = \begin{pmatrix} \sin(\theta) \\ -\cos(\theta) \end{pmatrix} \quad Y^T = \begin{pmatrix} \sin(\theta) & -\cos(\theta) \end{pmatrix}$$

$$Y Y^T = \begin{pmatrix} \sin^2(\theta) & -\sin(\theta) \cos(\theta) \\ -\sin(\theta) \cos(\theta) & \cos^2(\theta) \end{pmatrix}$$

$$A = X X^T + Y Y^T$$

$$= \begin{pmatrix} \cos^2(\theta) & \cos(\theta) \sin(\theta) \\ \cos(\theta) \sin(\theta) & \sin^2(\theta) \end{pmatrix} + \begin{pmatrix} \sin^2(\theta) & -\sin(\theta) \cos(\theta) \\ -\sin(\theta) \cos(\theta) & \cos^2(\theta) \end{pmatrix}$$

$$= \begin{pmatrix} \cos^2(\theta) + \sin^2(\theta) & \cos(\theta) \sin(\theta) - \sin(\theta) \cos(\theta) \\ \cos(\theta) \sin(\theta) - \sin(\theta) \cos(\theta) & \sin^2(\theta) + \cos^2(\theta) \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

```

import numpy as np
theta = np.pi/6

X = np.array([[np.cos(theta)],
               [np.sin(theta)]])

Y = np.array([[np.sin(theta)],
               [-np.cos(theta)]])

XXT = np.dot(X, X.T)
YYT = np.dot(Y, Y.T)

A = XXT + YYT

print("A = XXT + YYT:\n", A)

```

```

A = XXT + YYT:
[[1. 0.]
 [0. 1.]]

```

Analytical and python programmed results are same.

(b)

```

import numpy as np

def check_symmetric(A):
    if A.shape[0] != A.shape[1]:
        return False
    return np.array_equal(A, A.T)

A = np.array([[2, 1, -3],
               [1, 2, -3],
               [-3, -3, -3]])

print ("Matrix A: \n", A)
print ("Is a symmetric?", check_symmetric(A))

```

```

Matrix A:
[[ 2  1 -3]
 [ 1  2 -3]
 [-3 -3 -3]]
Is a symmetric? True

```

If C is a symmetric matrix,

$$C = C^T$$

$$C = \begin{pmatrix} 2 & 1 & -3 \\ 1 & 2 & -3 \\ -3 & -3 & -3 \end{pmatrix}$$

$$C^T = \begin{pmatrix} 2 & 1 & -3 \\ 1 & 2 & -3 \\ -3 & -3 & -3 \end{pmatrix}$$

$C = C^T$ So this is a symmetric matrix.

So, the results are same.

(c)

```
import numpy as np

alpha = np.pi/3
beta = np.pi/4
delta = np.pi/4
gamma = 3*np.pi/2

def exp(theta):
    return np.cos(theta)+1j*np.sin(theta)

A1 = np.array([[exp(-beta/2), 0],
               [0, exp(beta/2)]])

A2 = np.array([[np.cos(gamma/2), -np.sin(gamma/2)],
               [np.sin(gamma/2), np.cos(gamma/2)]])

A3 = np.array([[exp(-delta/2), 0],
               [0, exp(delta/2)]])

global_phase = exp(alpha)
U = global_phase*A1@A2@A3

print("Matrix U is: ", U)
```

```
Matrix U is: [[-0.6830127 -0.1830127j -0.35355339-0.61237244j]
 [ 0.35355339+0.61237244j  0.1830127 -0.6830127j ]]
```

3. I.

(d) Eigenvalues and eigenvectors are fundamental concepts in linear algebra that provide deep insights into the behavior of linear transformations

(e) The characteristic equation is a polynomial equation derived from a square matrix that helps determine the matrix's eigenvalues. Specifically,

(f) In linear algebra, eigenvalues and eigenvectors provide geometric insights into how a matrix transforms space. An eigenvector is a vector that, when transformed by the matrix, only changes in length (scales) but not direction, meaning it stays on the same line. The eigenvalue is the factor by which the eigenvector is scaled during this transformation.